

Written reflection of the two advanced topics

RNGChess – Georgi Yordanov, Nikola Shtinkov

Topic 1 – Software Engineering

We use git and GitHub for our combined workflow.

Why you chose the topic and how it is relevant to your project – We chose Software Engineering because we are developing RNGChess as a team. Git and GitHub help us work on the same project without losing each other's changes.

What you learned through your research, including the main concepts or principles – We learned how version control works through commits, branches, merging, and resolving conflicts.

How you applied the topic in your project – We use Git and GitHub to store our project, combine our work, and keep track of different versions.

What you learned from the implementation, including challenges, decisions, and impact on your project – We learned that using Git correctly makes teamwork much easier, although merge conflicts can happen when we edit the same code.

Topic 2 – Design & Architecture

We use Advanced Object-Oriented Design Patterns to make our code much more modular, and we also use well known methods from User Experience & Game Design.

Why you chose the topic and how it is relevant to your project – Since RNGChess contains many different systems, such as chess

pieces, movement rules, dice mechanics, turns, and the graphical interface it is very important to have a good code structure so it does not become difficult to manage.

What you learned through your research, including the main concepts or principles - We learned about advanced object-oriented design and concepts such as inheritance, encapsulation, polymorphism, and separation of responsibilities. We also researched User Experience and Game Design principles, especially how to clearly communicate game information to the player.

How you applied the topic in your project - We separated different responsibilities into different classes, for example separate classes for chess pieces and game systems. We also designed the interface to clearly show information such as the current player, dice result, and remaining moves.

What you learned from the implementation, including challenges, decisions, and impact on your project - The main challenge was deciding how much functionality should be separated into different classes without making the project unnecessarily complicated. The modular structure made our code easier to understand, test, debug, and extend. The UX principles also helped make the RNG mechanics clearer for the player.